CSCI 112 Assignment 10 – Comparators, Generic Bubble Sort and `qsort()`

Points: 10

Automated tests will run on GitHub Actions after every push. Check the Actions tab to see which tests pass and which fail.

The tests also run with AddressSanitizer. A memory leak or an access outside an array will fail the whole run, even if every single assertion passes.

Instructions

1. **Fork** the repository on GitHub - `https://github.com/jpach-cs/112-ass10-template`.
2. Clone your forked repository to your local machine.
3. Open the file `code.h` and read it carefully. It is already complete: it contains the `Comparator` type, the `Student` struct and the prototypes of all 21 functions, each with a short description of what the function must do. **Do not change any signature** — the automated tests rely on them.
4. Open the file `code.c`.
    1. The file is a compilable skeleton: every function exists, has detailed notes above it, and contains a `// TODO` placeholder. Replace the placeholder (and the `(void)...;` lines under it, which only silence "unused parameter" warnings) with your implementation. `<stdlib.h>` (for `qsort`, `malloc`, `free`) and `<string.h>` (for `strcmp`, `memcpy`) are already included.
    2. Implement every function listed below, **in the given order** — later functions reuse the earlier ones.
        - Read the description of each function carefully.
        - Use ONLY the standard library (`stdlib.h`, `string.h`, `stdbool.h`, `stddef.h`).
        - Use the **exact behavior** described. The tests check the return values, the order of elements, and even how many times your sort calls the comparator.
    3. Test your implementation locally using `main.c` and compile it with the sanitizer (see "Local Testing").
5. Once all functions are complete — save your changes, then **commit** and **push** to your repository.
6. Go to the **Actions** tab on GitHub and check the test results.

Learning Objectives

After completing this lab, you will:

- Write **comparator functions** with the signature `int (*)(const void *, const void *)`

- Pass functions as parameters (callbacks) – the same idea you used in `apply_operation()`
- Work with `void *` pointers: casting, and pointer arithmetic with an element size
- Write a **generic** algorithm that sorts any type of data, just like `qsort()`
- Understand the difference between an array of fixed-size strings (`char[N][LEN]`) and an array of pointers (`char *[N]`)
- Use the built-in `qsort()` correctly (base, count, **element size**, comparator)
- Recognize and avoid typical comparator bugs (overflow, truncation to `int`)
- Combine sorting with dynamic memory (sorted copies) without leaks

---

The plan: a snowball

Part	Topic	Functions	Suggested time
1	Comparators for `int`, `float`, strings	8	~35 min
2	Your own generic sort: `swap_elements`, `bubble_sort`, `is_sorted_array`	3	~40 min
3	Using `qsort()` and the comparators	10	~45 min

Do **not** skip ahead. Part 2 sorts with the comparator *you* will pass in, and Part 3 reuses the comparators from Part 1 and the helpers from Part 2.

General rules

1. A comparator **never modifies** the elements it compares and has no side effects.
2. A comparator returns **any** negative number, zero, or any positive number. It does not have to return exactly `-1` / `1`. The tests only look at the **sign**.
3. Functions taking pointers must survive `NULL` arguments exactly as described (no crash).

4. A function that allocates memory must free everything it does not return. No leaks.
5. In functions that work with `void *`, convert to `unsigned char *` before you do pointer arithmetic. Never rely on `void *` arithmetic (it is not standard C).
6. Sizes and counts are `size_t`. The size of an element is always passed or computed with `sizeof` — never hard-coded.

Part 1 – Comparators

#	Function	Elements of the array are…
1	`int compare_int_asc(const void *a, const void *b)`	`int`
2	`int compare_int_desc(const void *a, const void *b)`	`int`
3	`int compare_float_asc(const void *a, const void *b)`	`float`
4	`int compare_float_desc(const void *a, const void *b)`	`float`
5	`int compare_str_asc(const void *a, const void *b)`	`char[LEN]` (whole strings stored in the array)
6	`int compare_str_desc(const void *a, const void *b)`	`char[LEN]`
7	`int compare_strptr_asc(const void *a, const void *b)`	`char *` (pointers to strings)

#	Function	Elements of the array are…
8	`int compare_strptr_desc(const void *a, const void *b)`	`char *`

1. `compare_int_asc` – Receives pointers to **two ints** (as `const void *`). Returns a negative number if the first is smaller, `0` if they are equal, and a positive number if the first is larger. It must be correct for **all** ints, including `INT_MIN` and `INT_MAX`.
2. `compare_int_desc` – The same, but larger numbers come first.
3. `compare_float_asc` – The same as 1 for `float`. It must tell apart values that differ by less than 1 (for example `1.5` and `1.2`).
4. `compare_float_desc` – The same as 3, but larger numbers come first.
5. `compare_str_asc` – Each element of the array is a **whole string** stored inside the array, so the pointer you receive points to the first character of that string. Compare the **whole strings** (`strcmp`), ascending. The order is by character codes, so `"Zebra"` comes **before** `"apple"` (`'Z'` is 90, `'a'` is 97).
6. `compare_str_desc` – The same, but reversed.
7. `compare_strptr_asc` – Each element of the array is a **pointer to a string** (`char *`). The pointers you receive point **to those pointers** (they are really `char **`). Get the string pointer out of the element first, then compare the strings (`strcmp`), ascending.
8. `compare_strptr_desc` – The same, but reversed.

Part 2 – Your own generic sort

#	Function	Key concept
9	`void swap_elements(void *a, void *b, size_t size)`	swapping raw bytes
10	`void bubble_sort(void *base, size_t count, size_t size, Comparator compare)`	a `qsort()` look-alike
11	`bool is_sorted_array(const void *base, size_t count, size_t size, Comparator compare)`	checking an order

9. `swap_elements` – Exchanges the contents of two blocks of `size` bytes, **byte by byte** (no `malloc`, no helper array). Does nothing if `a` or `b` is `NULL`, if both point to the same block, or if `size` is `0`.
10. `bubble_sort` – A generic bubble sort with the **same parameters as `qsort()`**: the array `base` has `count` elements, each `size` bytes long, ordered by `compare`. It must work for **any** element type.
    - Does nothing if `base` or `compare` is `NULL`, `size` is `0`, or `count < 2`.
    - The address of element `i` is `(unsigned char *)base + i * size`.
    - **Pass number `k`** (`k = 0, 1, 2, …`) compares only the first `count - 1 - k` pairs of neighbours; the last `k` elements are already in place.
    - Two neighbours are swapped **only** when `compare(left, right) > 0`. Because equal elements are never swapped, the sort is **stable**.
    - The sort stops after a pass in which **nothing was swapped**.
    - Consequently an already sorted array needs exactly `count - 1` comparisons, and a reversed array needs exactly `count * (count - 1) / 2`. The tests count the calls of your comparator.
11. `is_sorted_array` – Returns `true` if `compare(element i, element i + 1) <= 0` for every pair of neighbours. An array with fewer than 2 elements is sorted. Returns `false` if `compare` is `NULL` or `size` is `0`, or if `base` is `NULL` while `count >= 2`.

Part 3 – Using `qsort()` and the comparators

The `Student` type is defined in `code.h`:

```c
#define NAME_LEN 16

typedef struct
{
    char name[NAME_LEN];
    int age;
    float score;
} Student;
```

#	Function	Key concept
12	`void sort_ints(int *arr, size_t count, bool ascending)`	`qsort` + a comparator chosen at run time
13	`void sort_floats(float *arr, size_t count, bool ascending)`	the same for `float`

#	Function	Key concept
14	`void sort_names(char names[][NAME_LEN], size_t count, bool ascending)`	element size = `NAME_LEN`
15	`void sort_string_pointers(char **strs, size_t count, bool ascending)`	element size = `sizeof(char *)`
16	`int *sorted_copy(const int *arr, size_t count)`	`malloc` + `memcpy` + `qsort`
17	`bool find_median(const int *arr, size_t count, float *out)`	reuse + `free`
18	`bool has_duplicates(const int *arr, size_t count)`	sorting makes duplicates neighbours
19	`int compare_students_by_age(const void *a, const void *b)`	a comparator for a struct
20	`int compare_students_ranking(const void *a, const void *b)`	a comparator with two keys
21	`void rank_students(Student *students, size_t count)`	`qsort` with your own comparator

12. **`sort_ints`** – Sorts an array of `int` with the built-in `qsort()`: ascending if `ascending` is `true`, otherwise descending. Use the comparators from Part 1. Does nothing if `arr` is `NULL` or `count < 2`.
13. **`sort_floats`** – The same for an array of `float`.
14. **`sort_names`** – Sorts an array of fixed-size names (`NAME_LEN` bytes each) with `qsort()`. Think about what the **element size** is. Does nothing if `names` is `NULL` or `count < 2`.
15. **`sort_string_pointers`** – Sorts an array of `char *` with `qsort()`. Only the pointers move; the strings they point to are not changed. Does nothing if `strs` is `NULL` or `count < 2`.
16. **`sorted_copy`** – Returns a **new heap block** with the elements of `arr` sorted ascending. The original array is **not modified**. The caller owns (and frees) the result. Returns `NULL` if `arr` is `NULL`, `count` is `0` or the allocation fails.
17. **`find_median`** – Stores the median in `*out` and returns `true`.
    - Odd `count`: the middle value of the sorted data.
    - Even `count`: the **average** of the two middle values (for `{4, 1, 3,`

`2}` the median is `2.5`). Add them as floats, so that two large ints cannot overflow.
    - The array is not modified, and the copy you made must be freed before you return.
    - Returns `false` and leaves `*out` untouched if `arr` is `NULL`, `count` is `0`, `out` is `NULL` or an allocation fails.
18. **`has_duplicates`** – Returns `true` if some value occurs at least twice. The array is not modified; free what you allocate. Returns `false` if `arr` is `NULL` or `count < 2` (or if an allocation fails).
19. **`compare_students_by_age`** – A comparator for an array of `Student`: younger students first. Students with the same age compare as equal.
20. **`compare_students_ranking`** – A comparator that builds a ranking: **higher score first**; students with the same score are ordered by **name, ascending**; same score and same name compare as equal.
21. **`rank_students`** – Sorts an array of `Student` into the ranking order using `qsort()` and `compare_students_ranking`. Does nothing if `students` is `NULL` or `count < 2`.

Background

Function pointers as callbacks

You already passed a function to another function in `apply_operation(arr, 4, add)`. Sorting works the same way: the sorting code does not know how to compare your data, so you **hand it a comparator**.

```
typedef int (*Comparator)(const void *a, const void *b);
```

`Comparator` is a name for "pointer to a function that takes two `const void *` and returns an `int`". It is exactly the type that `qsort()` expects.

The comparator contract

A comparator receives **pointers to two elements** of the array (not the elements themselves) and returns:

Return value	Meaning
negative	`*a` must come **before** `*b`
`0`	they are equal (any order is fine)
positive	`*a` must come **after** `*b`

Because the arguments are `const void *`, you must first cast them to the right pointer type, then dereference:

```c
int compare_int_asc(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}
```

`(x > y) - (x < y)` works because a comparison gives `1` or `0`: `(5 > 3) - (5 < 3)` is `1 - 0 = 1`, `(3 > 5) - (3 < 5)` is `0 - 1 = -1`, and equal values give `0 - 0 = 0`.

Two classic comparator bugs

```c
/* BUG 1: subtraction can overflow */
return x - y;                   /* INT_MAX - (-1) overflows: undefined behavior */

/* BUG 2: truncation to int */
return (int)(fx - fy);          /* 1.5f - 1.2f = 0.3, (int)0.3 = 0 -> "equal"! */
```

Descending order for free

You do not need a second implementation. Swapping the arguments reverses the order:

```c
int compare_int_desc(const void *a, const void *b)
{
    return compare_int_asc(b, a);
}
```

`qsort()`

```c
void qsort(void *base, size_t count, size_t size,
           int (*compare)(const void *, const void *));
```

Parameter	Meaning
`base`	pointer to the first element
`count`	number of elements
`size`	size of **one element in bytes**
`compare`	your comparator

```c
int numbers[] = {5, 2, 9, 1};
qsort(numbers, 4, sizeof(int), compare_int_asc);
```

Use `sizeof *arr` (or `sizeof(type)`) for the element size — never a hard-coded number. A wrong size is the most common `qsort()` mistake: the array is sorted "in pieces" and the result looks random.

`qsort()` is **not guaranteed to be stable**: equal elements may end up in a different order than they had before sorting. Your `bubble_sort` **is** stable.

Two kinds of string arrays

```
char names[3][NAME_LEN] = {"pear", "apple", "fig"};     /* strings stored IN the array */
char *strs[3]           = {"pear", "apple", "fig"};     /* array of POINTERS to strings */
```

```
names (3 * NAME_LEN bytes):                strs (3 pointers):

 p e a r \0 ...     element 0                       pear

 a p p l e \0 ...   element 1                       apple

 f i g \0 ...       element 2                       fig

element size = NAME_LEN                    element size = sizeof(char *)
comparator gets: a pointer to the          comparator gets: a pointer to the
first character (char *)                   pointer (char **)
```

So `compare_str_asc` is `strcmp((const char *)a, (const char *)b)`, while `compare_strptr_asc` must first dereference: `strcmp(*(const char * const *)a, *(const char * const *)b)`. Using the wrong one compiles, but crashes or sorts garbage.

Bubble sort

Bubble sort compares neighbours and swaps them when they are in the wrong order. After each pass the largest remaining element has "bubbled up" to its final place at the end.

```
{5, 2, 9, 1}   pass 0 (3 pairs):  5>2 swap -> 2 5 9 1 ; 5<9 ok ; 9>1 swap -> 2 5 1 9
               pass 1 (2 pairs):  2<5 ok ; 5>1 swap -> 2 1 5 9
               pass 2 (1 pair):   2>1 swap -> 1 2 5 9
               done: 3 + 2 + 1 = 6 comparisons
```

- The last `k` elements are already in place after `k` passes, so pass `k` compares only `count - 1 - k` pairs.
- If a whole pass makes no swap, the array is sorted and you can stop.
- It needs about `count`^{2}`/2` comparisons, so it is **slow for large arrays**. We write it to understand sorting; in real programs use `qsort()`.

A generic swap

You cannot write `T tmp = *a;` when you do not know `T`. Instead you swap **bytes**:

```
unsigned char *pa = a;
unsigned char *pb = b;
```

```c
for (size_t i = 0; i < size; i++)
{
    unsigned char tmp = pa[i];
    pa[i] = pb[i];
    pb[i] = tmp;
}
```

The same trick gives you the address of element `i` of any array: `(unsigned char *)base + i * size`.

Why sort a copy?

`qsort()` sorts **in place**. A function that only wants to *look* at sorted data (a median, duplicates) must not reorder the caller's array. So it copies first:

```c
int *copy = malloc(count * sizeof *copy);
memcpy(copy, arr, count * sizeof *copy);
qsort(copy, count, sizeof *copy, compare_int_asc);
/* ... use copy ... */
free(copy);
```

Only for testing

Local Testing (Sandbox)

Compile with the sanitizer – it reports leaks and accesses outside an array:

```
gcc -std=c99 -g -Wall -Wextra -fsanitize=address,undefined main.c code.c -o main
./main
```

Create test cases in main.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include "code.h"

int main()
{
    // Part 1 + 3: qsort with the comparators
    int numbers[] = {5, 2, 9, 1, 7};
    sort_ints(numbers, 5, true);
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }
    printf("\n");
```

```c
    char names[4][NAME_LEN] = {"pear", "Zebra", "apple", "fig"};
    sort_names(names, 4, true);
    for (int i = 0; i < 4; i++)
    {
        printf("%s ", names[i]);
    }
    printf("\n");

    // Part 2: your own generic sort works for any type
    float values[] = {2.5f, -1.0f, 1.2f, 1.5f};
    bubble_sort(values, 4, sizeof(float), compare_float_asc);
    for (int i = 0; i < 4; i++)
    {
        printf("%.2f ", values[i]);
    }
    printf("\n");
    printf("sorted? %d\n", is_sorted_array(values, 4, sizeof(float), compare_float_asc));

    // Part 3: median and duplicates
    int data[] = {4, 1, 3, 2};
    float median;
    if (find_median(data, 4, &median))
    {
        printf("median = %.2f\n", median);
    }
    printf("duplicates: %d\n", has_duplicates(data, 4));

    // Part 3: students
    Student students[3] = {{"Zoe", 20, 90.0f}, {"Adam", 22, 85.5f}, {"Bob", 21, 90.0f}};
    rank_students(students, 3);
    for (int i = 0; i < 3; i++)
    {
        printf("%d. %s (%.1f)\n", i + 1, students[i].name, students[i].score);
    }

    return 0;
}
```

Hints: - If your `bubble_sort` gives the wrong result for `float` but the right one for `int`, check how you compute the address of element `i` — it must use `size`, not `sizeof(int)`. - If `sort_names` scrambles the letters, the element size you passed to `qsort()` is wrong. - Write each function, compile, and test it **before** moving to the next one.